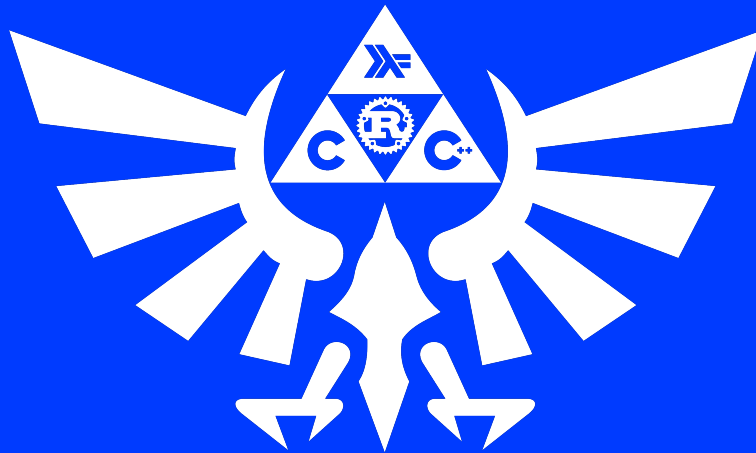




DAY 12

< A DEEP DIVE INTO THE RUST LANGUAGE />



DAY 12



Project to deliver: Cargo.toml
language: Rust



- ✓ The totality of your source files, except all useless files (binary, temp files, obj files,...), must be included in your delivery.
- ✓ All the bonus files (including a potential specific Makefile) should be in a directory named bonus.

Exercise 00 - Float Recognition



Turn-in : `float_recognition.rs` in a folder named `ex00`

The Union wants you to continue your learning in the Rust Language and become a great [Rustacean](#). Therefore here is a trivial exercise that you'll surely be able to do easily !



The only function you can use to check if something is a number is the function `is_digit`

Do a function with the following prototype : `pub fn is_number(number: &str) -> bool` that return `true` when the string given as parameter is a number (positive or negative, integer or floating point). You do not *need* to check numbers written as octal, binary, hexadecimal or in [E-notation](#). Here are a few examples:

```
fn main() {
    println!("Is \"42.42\" a number ? {}", is_number("42.42"));
    println!("Is \"42.24.42\" a number ? {}", is_number("42.24.42"));
    println!("Is \"yeet\" a number ? {}", is_number("yeet"));
    println!("Is \"\t123.456\t\" a number ? {}", is_number("\t123.456\t"));
    println!("Is \"123 .456\" a number ? {}", is_number("123 .456"));
    println!("Is \"-0.4\" a number ? {}", is_number("-0.4"));
    println!("Is \"-.8\" a number ? {}", is_number("-.8"));
    println!("Is \"8.-6\" a number ? {}", is_number("8.-6"));
    println!("Is \"-8-.6\" a number ? {}", is_number("-8-.6"));
}
```

```
Terminal
$> cargo run
Is "42.42" a number ? true
Is "42.24.42" a number ? false
Is "yeet" a number ? false
Is "\t123.456\t" a number ? true
Is "123 .456" a number ? false
Is "-0.4" a number ? true
Is "-.8" a number ? false
Is "8.-6" a number ? false
Is "-8-.6" a number ? false
```

The Union is very careful about the code written to embark on rocket ships and wants you to test it thoroughly ! According to the test protocol, you are required to test your float recognition function. Write unit tests for the previous function that can be tested with the command `cargo test`. At least five different tests are expected for this testing protocol to be enough.

Exercise 01 - The Null Paradox



Turn-in : `null_paradox.rs` in a folder named `ex01`

The Union's history books talk about "The Null Paradox". Centuries ago, navigation systems used a value called `NULL` to represent "no destination".

The problem? `NULL` looked exactly like the memory address `0x00`.

In 2099, the autopilot of the flagship "Titanic II" tried to read address `0x00`, panicked, and steered the ship directly into a black hole.

Since then, `NULL` is forbidden. The Union uses the `Option<T>` enum to wrap values. A value is either `Some(data)` or `None`. It forces engineers to handle the "nothingness" explicitly.

Your task is to update the navigation module to strictly use `Option` logic.

Instructions

You must define a struct and implement three functions to handle navigation data.

Define a struct named `Coordinates`:

```
pub struct Coordinates {  
    pub x: i32,  
    pub y: i32,  
    pub z: i32,  
}
```

Write a function that acts as a mock database.

```
pub fn get_place(name: &str) -> Option<Coordinates>
```

- ✓ If `name` is `"Earth"`, return `Some(Coordinates)` with values `(0, 0, 0)`.
- ✓ If `name` is `"Mars"`, return `Some(Coordinates)` with values `(15, 22, 110)`.
- ✓ If `name` is `"Jupiter"`, return `Some(Coordinates)` with values `(88, 12, -4)`.
- ✓ For any other name, the database knows nothing. Return `None`.

Then, write a function that safely analyzes the result.

```
pub fn analyze_transport_method(option: Option<Coordinates>) -> String
```

You must use **Pattern Matching** (`match`) to handle both cases:

- ✓ If it is `Some(coords)`: Return the string `"Teleporting to <x>,<y>,<z>"`.
- ✓ If it is `None`: Return the string `"No destination available"`.

Sometimes, engineers are lazy. They are sure the value exists, so they force the wrapper open.

```
pub fn risky_teleport(option: Option<Coordinates>) -> String
```

- ✓ This function must simply return the string `"Teleporting to <x>,<y>,<z>"`.
- ✓ You are **FORBIDDEN** to use `match` or `if`.
- ✓ You must use the `.unwrap()` method on the option to extract the coordinates directly.

Think about it: What happens if `risky_teleport` receives `None`? This is the core of the paradox.

Example

Here is a `main` function to test your code. Notice how the last call will crash your program. This is expected behavior for this exercise.

```
fn main() {
    let earth_coords = get_place("Earth");
    let mars_coords = get_place("Mars");
    let void_coords = get_place("Black Hole");

    println!("--- Safe Analysis ---");
    println!("Earth: {}", analyze_transport_method(earth_coords));
    println!("Void: {}", analyze_transport_method(void_coords));
    println!("\n--- Risky Teleport ---");
    println!("Mars: {}", risky_teleport(get_place("Mars")));
    println!("Now trying to teleport to the Void...");
    println!("Void: {}", risky_teleport(get_place("Black Hole")));
}
```

```
Terminal
$> cargo run
--- Safe Analysis ---
Earth: Teleporting to 0,0,0
Void: No destination available
--- Risky Teleport ---
Mars: Teleporting to 15,22,110
Now trying to teleport to the Void...
thread 'main' panicked at 'called Option::unwrap() on a None value', src/null_paradox.rs:XX:XX
```

Exercise 02 - The Food Court



Turn-in : `food_court.rs` in a folder named `ex02`

The Union cares about the diet of its astronauts. A specific automated kitchen has been installed in the cafeteria. You need to write the backend logic to check if ingredients are available to cook meals.

However, the database is unstable, and some experimental ingredients stored in the kitchen are highly volatile. **Be careful what you order.**

You must use the following structure to represent a meal plan:

```
pub struct Recipe {  
    pub name: String,  
    pub ingredients: Vec<String>,  
}
```

You must implement **exactly** these four functions to make the system work.

1. The Database

```
pub fn get_ingredient_quantity(ingredient: &str) -> Result<usize, String>
```

This function checks the hardcoded stock.

- ✓ **"Space Bacon"**: 42 units.
- ✓ **"Dehydrated Eggs"**: 120 units.
- ✓ **"Soy Milk"**: 5 units.
- ✓ **"Antimatter"**: **DANGER**. The containment field is weak. If this is requested, the program must **crash immediately** using the `panic!` macro with the message: `"BOOM! The kitchen exploded!"`.
- ✓ **"Recycled Water"**: Returns an error with the message: `"Machine broken"`.
- ✓ **Anything else**: Returns an error with the message: `"Food not found"`.

2. The Menu Parser

```
pub fn resolve_recipe(name: &str) -> Result<Recipe, String>
```

This function converts a meal name into a `Recipe` struct.

- ✓ **"Space Omelette"**: needs "Space Bacon" and "Dehydrated Eggs".
- ✓ **"Corpse Reviver"**: needs "Soy Milk" and "Recycled Water".
- ✓ **"Quantum Brownie"**: needs "Space Bacon" and "Antimatter".
- ✓ **Anything else**: Returns an error with the message "Meal not on menu".

3. The Stock Checker

```
pub fn check_availability(recipe: &Recipe) -> Result<String, String>
```

This function takes a reference to a `Recipe`. It must iterate through the recipe's `ingredients` and check the quantity of **each one** using `get_ingredient_quantity`.

- ✓ If **all** ingredients are available, return `Ok` with the message: "All ingredients available".
- ✓ If **any** ingredient triggers an error, stop immediately and return that error.

4. The Master Chef

```
pub fn cook_meal(order: &str) -> Result<String, String>
```

This is the main entry point. It chains the logic:

1. Resolve the recipe name.
2. Check if ingredients are available.
3. If everything is good, return `Ok` with the message: "The <meal_name> is ready!".
4. If any step fails, propagate the error.



You are **FORBIDDEN** to use `.unwrap()` or `.expect()` (unless you want to create the crash yourself inside the panic case). You must handle the `Result` types properly using `match` or the `?` operator.

Example

Here is a `main` function to test your code. Notice what happens with the last order.

```
fn main() {
    println!("--- Order 1: Space Omelette ---");
    match cook_meal("Space Omelette") {
        Ok(msg) => println!("Success: {}", msg),
        Err(e) => println!("Error: {}", e),
    }

    println!("\n--- Order 2: Corpse Reviver ---");
    match cook_meal("Corpse Reviver") {
        Ok(msg) => println!("Success: {}", msg),
        Err(e) => println!("Error: {}", e),
    }

    println!("\n--- Order 3: Quantum Brownie (Experimental) ---");
    // This part should technically never print the Result because the program will crash
    // before.
    match cook_meal("Quantum Brownie") {
        Ok(msg) => println!("Success: {}", msg),
        Err(e) => println!("Error: {}", e),
    }

    println!("This line will never be printed if the kitchen explodes.");
}
```

Terminal

```
$> cargo run
--- Order 1: Space Omelette ---
Success: The Space Omelette is ready!
--- Order 2: Corpse Reviver ---
Error: Machine broken
--- Order 3: Quantum Brownie (Experimental) ---
thread 'main' panicked at 'BOOM! The kitchen exploded!', src/food_court.rs:25:13
```

Exercise 03 - The Space Enigma



Turn-in : `space_enigma.rs` in a folder named `ex03`

The Union has intercepted a stream of communications from an unknown civilization. You have been tasked to build the **Space Enigma**, a machine capable of categorizing, authorizing, decoding, and **analyzing** these messages.

1. The Data Models

You must define the following data structures.

An `enum` named `Message`:

1. `ClearString(String)`: A standard text message.
2. `DigitalCode(u32)`: A raw numeric code.
3. `Encrypted(String, u32)`: A message content + an encryption key ID.

An `enum` named `SecurityClass`:

1. `Civilian`: Open to everyone.
2. `Diplomatic`: Restricted.
3. `Military`: Highly classified.

A `struct` named `Transmission`:

```
pub struct Transmission {
    pub message: Message,
    pub security: SecurityClass,
    pub signal_strength: u8, // 0 to 255
}
```

2. The Engine

You must implement a single function that processes a transmission.

```
pub fn solve_enigma(transmission: Transmission, daily_key: u32) -> Result<String, String>
```

You must implement the following 4 phases in order. If any phase fails, return the error immediately.

Phase 1: Signal Integrity

- ✓ If `signal_strength` is **0**, return an error with the message: "Signal lost".
- ✓ If `signal_strength` is strictly below **20**, return an error with the message: "Signal too weak".

Phase 2: Security Clearance

- ✓ **Military** messages require the `daily_key` to match the value **4242**. If not, return an error with the message: "Security Breach: Invalid Key".
- ✓ Other classes are unrestricted.

Phase 3: Decoding

Decode the message into a temporary `String`:

1. `ClearString`: Take the string as is. If it is "Bomb", return an error with the message: "Panic: Bomb detected".
2. `DigitalCode`:
 - **404** becomes "Not Found".
 - **200** becomes "OK".
 - Any other number becomes "Code: <number>".
3. `Encrypted(content, key_id)`:
 - If `key_id` matches `daily_key`, the string is the content.
 - If not, return an error with the message: "Encryption Error: Wrong Key".

Phase 4: The Sanity Check

Now that you have the decoded string, you must analyze its content to ensure it's not generated by an AI bot. Calculate a **Complexity Score**:

- ✓ Iterate through every character of the string.
- ✓ Add **+1** for every alphabetic character (a-z, A-Z).
- ✓ Subtract **-1** for every numeric digit (0-9).
- ✓ Ignore other symbols (spaces, punctuation).

Final Decision:

- ✓ If the score is **strictly negative** (more numbers than letters), return an error with the message: "Artificial Noise".
- ✓ Otherwise, return `ok` with the decoded string.



You will need to iterate over the string. So... think about Rust iterators!

Example

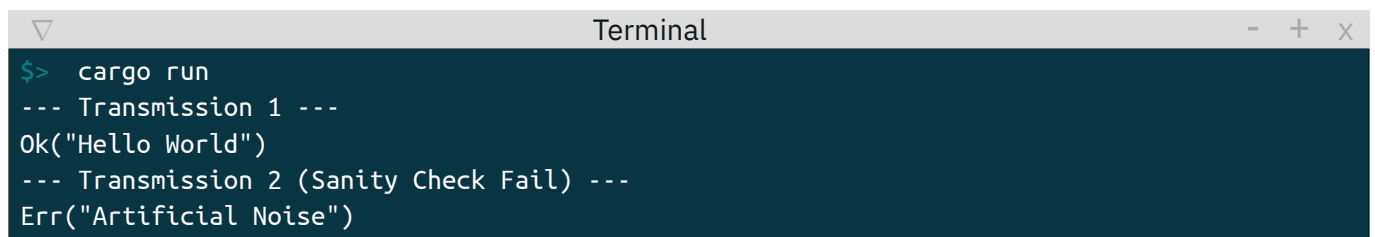
```
fn main() {
    let key = 4242;

    let t1 = Transmission {
        message: Message::ClearString(String::from("Hello World")),
        security: SecurityClass::Civilian,
        signal_strength: 100,
    };

    let t2 = Transmission {
        message: Message::Encrypted(String::from("Target 89898989"), 4242),
        security: SecurityClass::Military,
        signal_strength: 80,
    };

    println!("--- Transmission 1 ---");
    println!("{:?}", solve_enigma(t1, key));

    println!("--- Transmission 2 (Sanity Check Fail) ---");
    println!("{:?}", solve_enigma(t2, key));
}
```



A terminal window titled "Terminal" with standard window controls (minimize, maximize, close). The terminal shows the output of the Rust program executed with `cargo run`. The output consists of two lines of dashes, the result of solving the first transmission, another two lines of dashes, and the result of solving the second transmission.

```
$> cargo run
--- Transmission 1 ---
Ok("Hello World")
--- Transmission 2 (Sanity Check Fail) ---
Err("Artificial Noise")
```

Exercise 04 - Survivability V2



Turn-in : `survivability.rs` in a folder named `ex04`

The Union would like you to do some adjustments on the work you did yesterday because it's not so good anymore now that we've tested it. Be careful, you're on thin ice !

First, you will rework a function from yesterday. It will now be `compute_survivable_days(needs: &ShipAssessment, nb_crew: u32)-> u32`. However, this time, a crew member needs 3.0 liters of water per day. You will use this struct as the first parameter:

```
pub struct ShipAssessment {
    pub water_quantity: f32,
    pub water_cycles: u32,
    pub food_quantity: f32,
}
```

Then you will do another function prototyped as follow : `study_ships(ships: &[ShipAssessment], rocks_to_visit : &[(String, u32)], rock: String, nb_crew: u32)-> StudyResult` to study the reachability of a list of planets or moons that we will call 'rocks' by our fleet of spaceships.

The function takes a list of ship characteristics as the first parameter, a list of tuple of rocks as the second parameter (kind of like a `dict` in python or a `map` in C++), a rock that will be tested and the number of crew as the last parameter. The result type is a struct:

The function should return the result for the **first ship** in the list that can successfully reach the destination. If no ship can reach the destination, return a result with `reachable: false` (you can use the index 0 by default).

```
pub struct StudyResult {
    pub rock_name: String,
    pub reachable: bool,
    pub ship_index: usize,
}
```

And here is an example of a main function and its output:

```
fn main() {
    let nb_crew : u32 = 3;
    let needs = ShipAssessment {
        water_quantity: 600.0,
        water_cycles: 25,
        food_quantity: 3020.0,
    };
    let ship1 = ShipAssessment {
        water_quantity: 16.0,
        water_cycles: 270,
        food_quantity: 230.0,
    };
    let ship2 = ShipAssessment {
        water_quantity: 100.0,
        water_cycles: 50,
        food_quantity: 250.0,
    };
    let ship3 = ShipAssessment {
        water_quantity: 600.0,
        water_cycles: 25,
        food_quantity: 3020.0,
    };
    let ships = [
        ship1,
        ship3,
        ship2,
    ];
    let days = compute_survivable_days(&needs, nb_crew);
    println!("Days: {}", days);

    let rocks_to_visit = [(String::from("Venus"), 137), (String::from("Mars"), 154), (
        String::from("Europa"), 2007)];
    let to_visit = String::from("YEET");
    let mut study: StudyResult = study_ships(&ships, &rocks_to_visit, to_visit, nb_crew);
    println!("Rock: {}, Reachable: {}, Index: {}", study.rock_name, study.reachable, study
        .ship_index);
    let to_visit = String::from("Europa");
    study = study_ships(&ships, &rocks_to_visit, to_visit, nb_crew);
    println!("Rock: {}, Reachable: {}, Index: {}", study.rock_name, study.reachable, study
        .ship_index);
    let to_visit = String::from("Venus");
    study = study_ships(&ships, &rocks_to_visit, to_visit, nb_crew);
    println!("Rock: {}, Reachable: {}, Index: {}", study.rock_name, study.reachable, study
        .ship_index);
}
```

Terminal

```
$> cargo run
Days: 1666
Rock: Error, Reachable: false, Index: 0
Rock: Europa, Reachable: false, Index: 1
Rock: Venus, Reachable: true, Index: 0
```

Exercise 05 - Global Inventory



Turn-in : `global_inventory.rs` in a folder named `ex05`

The Union receives cargo shipments from dozens of mining colonies every day. The data arrives as a raw list of orders. Each order contains the resource name, the quantity received, and the current market price per unit.

Your job is to consolidate all these separate shipments into a single **Global Inventory**. You need to calculate the total quantity and the total financial value for each resource.

You must use the following structure for the values in your HashMap:

```
pub struct Stock {  
    pub quantity: u32,  
    pub total_value: u32,  
}
```

The input data will be provided as a slice of tuples:

`(&str, u32, u32)` representing `(Name, Quantity, Price_Per_Unit)`.

Implement the function `consolidate_inventory`:

```
pub fn consolidate_inventory(orders: &[(&str, u32, u32)]) -> HashMap<String, Stock>
```

Iterate through the `orders`. For each order:

1. **Check** if the resource (the name) already exists in the map.
2. **If it's new:**
 - ✓ Insert a new entry.
 - ✓ `quantity` is the order's quantity.
 - ✓ `total_value` is `quantity * price_per_unit`.
3. **If it exists:**
 - ✓ **Update** the existing entry.
 - ✓ Add the new quantity to the existing `quantity`.
 - ✓ Add the calculated value (`quantity * price_per_unit`) to the existing `total_value`.

The logistic drones sometimes glitch and send orders with **0 quantity**.

- ✓ If you encounter an order with 0 quantity, **do not** add it to the map if it's new.
- ✓ If the item already exists in the map, a 0 quantity order does nothing (it changes nothing).



As always, remember that the input strings are borrowed (&str), but your HashMap must own its keys (String).

Example

```
fn main() {
    let orders = [
        ("Sapphire", 10, 500),
        ("Iron", 100, 5),
        ("Sapphire", 5, 600),
        ("Iron", 50, 5),
        ("Space Dust", 0, 1),
    ];

    let inventory = consolidate_inventory(&orders);

    println!("--- Global Inventory ---");

    for (name, stock) in &inventory {
        println!("{: Qty {}, Value {}", name, stock.quantity, stock.total_value);
    }

    if let Some(stock) = inventory.get("Sapphire") {
        if stock.quantity == 15 && stock.total_value == 8000 {
            println!("Sapphire stock is correct.");
        } else {
            println!("Sapphire stock is WRONG.");
        }
    }

    if inventory.contains_key("Space Dust") {
        println!("Error: Space Dust should not be in the inventory.");
    }
}
```

```
Terminal
$> cargo run
--- Global Inventory ---
Iron: Qty 150, Value 750
Sapphire: Qty 15, Value 8000
Sapphire stock is correct.
```

Exercise 06 - Flight Log



Turn-in : `flight_log.rs` in a folder named `ex06`

System Alert: The Main Traffic Control tower at Station 13 has suffered a catastrophic power failure. The live schedule is gone.

We only have the **Raw Dump** from the backup sensors. It is a messy list of strings, some corrupted, some valid.

Your mission is to write a library to rebuild the flight schedule. You must parse each line, filter out the corrupted data, and sort the valid flights by departure time.

You **must** use the following structures. Copy them into your code exactly as they are.

```
#[derive(Debug, PartialEq)]
pub enum FlightStatus {
    OnTime,
    Delayed(u32),           // Delay in minutes
    Cancelled(String),     // Reason for cancellation
}

#[derive(Debug)]
pub struct Flight {
    pub id: String,
    pub destination: String,
    pub time: u32,          // Time in 24h format (e.g., 1430 for 14:30)
    pub status: FlightStatus,
}
```

The Parsing Engine

Implement the function that converts a raw string line into a `Flight` struct.

```
pub fn parse_flight(line: &str) -> Result<Flight, String>
```

Format of a valid line: `"ID,DESTINATION,TIME,STATUS"`

Parsing Rules:

1. **Splitting:** The fields are separated by commas `,`. If a line does not have exactly 4 fields, return `Err("Bad format")`.
2. **Time:** Must be parsed as a `u32`. If invalid (e.g., "14h30"), return `Err("Bad time")`.
3. **Status:** The status string determines the Enum variant:
 - ✓ "ONTIME" → `FlightStatus::OnTime`
 - ✓ "DELAYED <number>" (e.g., "DELAYED 45") → `FlightStatus::Delayed(45)`. If the number is missing or invalid, return `Err("Bad delay")`.
 - ✓ "CANCELLED <reason>" (e.g., "CANCELLED Weather") → `FlightStatus::Cancelled("Weather")`.
 - ✓ Anything else → `Err("Unknown status")`.

The Schedule Builder

Implement the function that processes a whole list of logs.

```
pub fn build_schedule(raw_logs: Vec<&str>) -> Vec<Flight>
```

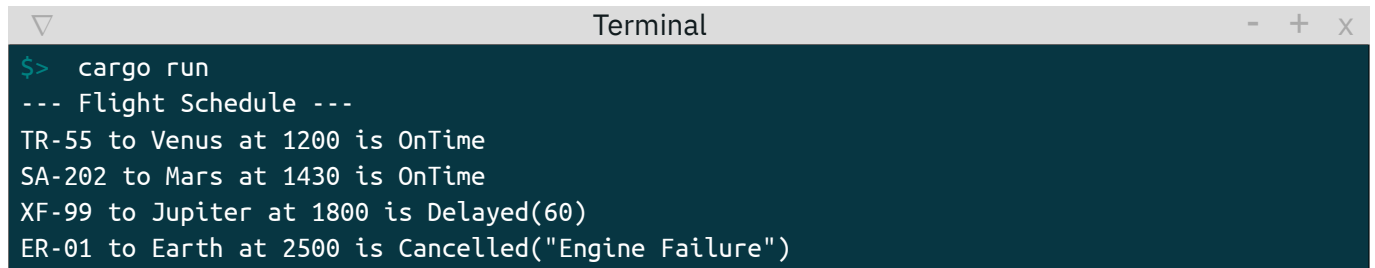
Logic:

1. Create a vector to store valid flights.
2. Iterate through the `raw_logs`.
3. For each log, call `parse_flight`.
 - ✓ If valid (`Ok`): Add the flight to your vector.
 - ✓ If invalid (`Err`): Ignore it (do not panic, just skip it).
4. **Sort** the resulting vector by `time` (Ascending: 0000 to 2359).
5. Return the clean, sorted vector.

Example

You can use this `main` function to test your library.

```
fn main() {  
    let logs = vec![  
        "SA-202,Mars,1430,ONTIME",  
        "XF-99,Jupiter,1800,DELAYED 60",  
        "BAD-LINE,Nowhere,0000",  
        "ER-01,Earth,2500,CANCELLED Engine Failure",  
        "TR-55,Venus,1200,ONTIME",  
        "ZZ-99,Pluto,invalid,ONTIME",  
    ];  
  
    let schedule = build_schedule(logs);  
  
    println!("--- Flight Schedule ---");  
    for f in schedule {  
        println!("{}", f.id, f.destination, f.time, f.status);  
    }  
}
```



```
Terminal  
$> cargo run  
--- Flight Schedule ---  
TR-55 to Venus at 1200 is OnTime  
SA-202 to Mars at 1430 is OnTime  
XF-99 to Jupiter at 1800 is Delayed(60)  
ER-01 to Earth at 2500 is Cancelled("Engine Failure")
```

Exercise 07 - Budget Analysis



Turn-in : Cargo project in `ex07` directory that could be compiled and run with `cargo run`

The Union needs you to create a program able to analyze the budget that has been allowed to the space research this year by the Ministry of Finance of The Union. Each year a new document is written by the Ministry of Finance, someone needs to read it and it costs time and effort. The Union wants that process to be automated.

Your program will take exactly three parameters, no more, no less:

- The budget file of the year N-1
- The budget file of the year N
- The department we want to analyse

A budget file is always composed of similar elements: a few lines of official text, a list of department and their budget and another few lines of official text. Each section is separated by a complete line of `=`. The Union wants you to know that the two lines of `=` will always be present so you won't have to check this and only the second section is useful in this case.

Each line of the list starts with the name of the department, then the budget they are allowed and a percentage of increase inside parenthesis. Yes, each year the Ministry of Finance write an increase in budget, even if there is a decrease. So you will need to calculate the increase or decrease yourself.

The Union can create and delete departments every year. If a department exist in the year N-1 but not in the year N you must write on the standard output `Department deleted this year, no comparisons possible !`. If it's the other way around you must write on the standard output `Department created this year, no comparisons possible !`. In both cases you must exit your program with a return value of 0.



You must not verify every line of the documents, only the line you are supposed to look at based on the third parameter of your program. If the Ministry of Finance issued a document where the syntax of the line is not respected, write on the error output `The document is probably not arrived yet...`, return 84 and never talk about it to anyone.

Here are some examples:

```
Terminal
$> cat 2024_budget
Ministry of Finance - 2024 budget for space research and exploration.
The Ministry of Finance of The Union is proud to announce that this year,
the budget of space and research exploration has been increased. Every
department has received more funds to research and explore space.
=====
Department of Launch and Lunch: 123.00 M (+70.93%)
Department of Fundamental Research: 100.06 M (+32.03%)
Department of Tests and Failures: 16.54 M (+6.03%)
Department of Alien Life Recognition: 399.08 M (+20.43%)
Department of Rocket Science: 224.98 M (+13.33%)
Department of Human Survivability: 46.63 M (+2.2%)
Department of Extra-Terrestrial Vehicle: 90.00 M (+53.62%)
Department of Sabers and Lasers: 14.18 M (+15.15%)
=====
This is an official document of the Ministry of Finance of The Union, do
not reproduce, do not edit, do not read too much, do not use as a coaster, do
not put in the microwave.
$> cat 2025_budget
Ministry of Finance - 2025 budget for space research and exploration.
The Ministry of Finance of The Union is proud to announce that this year,
the budget of space and research exploration has been increased. Every
department has received more funds to research and explore space.
=====
Department of Launch and Lunch: 125.00 M (+3.30%)
Department of Fundamental Research: 90.04 M (+14.80%)
Department of Tests and Failures: 14.72 M (+6.80%)
Department of Alien Life Recognition: 437.18 M (+74.00%)
Department of Rocket Science: 204.90 M (+13.33%)
Department of Human Survivability: 47.63 M (+31.05%)
Department of Extra-Terrestrial Vehicle: 87.93 M (+42.24%)
Department of Lunar Exploration: 78.04 M (+12.34%)
=====
This is an official document of the Ministry of Finance of The Union, do
not reproduce, do not edit, do not read too much, do not use as a coaster, do
not put in the microwave.
```

```
Terminal
$> cargo run 2024_budget 2025_budget "Tests and Failures"
This year the budget of the department of Tests and Failures has been lowered by -11.00%
$> echo $?
0
$> cargo run 2024_budget 2025_budget "Launch and Lunch"
This year the budget of the department of Launch and Lunch has been raised by 1.63%
$> echo $?
0
$> cargo run 2024_budget 2025_budget "Security in Space"
The document is probably not arrived yet...
$> echo $?
84
$> cargo run 2024_budget 2025_budget "Sabers and Lasers"
Department deleted this year, no comparisons possible !
$> echo $?
0
$> cargo run 2024_budget 2025_budget "Lunar Exploration"
Department created this year, no comparisons possible !
$> echo $?
0
```

v1

{EPITECH}